ByteMart Digital

THE COMPLETE GUIDE

Microservices

From Fundamentals to Professional Practice

A complete, example-driven guide to microservices — from decomposition, communication, and resilience to distributed data, observability, security, and deployment.

30	200+	14	8
Chapters	Code Examples	Visual Diagrams	Modules

bytemartdigital.com

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

Microservices: The Complete Guide

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
The information in this book is provided in good faith and for educational purposes. While every effort has been made to ensure accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from the use of the code or information contained herein. All code examples are provided "as is".

Trademarks
Kubernetes, Kafka, and other product names, logos, and brands referenced are the property of their respective owners and are used for identification purposes only. This guide is an independent educational work.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

1. From Monolith to Microservices 4
2. Core Principles & Characteristics 7
3. Service Decomposition 9
4. When Not to Use Microservices 12
5. Synchronous Communication 14
6. gRPC & Efficient RPC 17
7. Asynchronous Messaging 20
8. API Contracts & Versioning 23
9. Service Discovery 26
10. The API Gateway 29
11. Centralized Configuration 32
12. Client-Side Load Balancing 35
13. Failure in Distributed Systems 37
14. Circuit Breakers 39
15. Retries, Timeouts & Bulkheads 42
16. Rate Limiting & Backpressure 45
17. Database per Service 47
18. The Saga Pattern 50
19. Event-Driven Architecture & CQRS 53
20. Eventual Consistency 56
21. Centralized Logging 58
22. Distributed Tracing 60
23. Metrics & Health Monitoring 63
24. Securing Microservices 65
25. Containerization 67
26. Orchestration with Kubernetes 69
27. CI/CD & Deployment Strategies 72
28. Key Microservices Patterns 74
29. Testing Microservices 77
30. Anti-Patterns & Best Practices 80

MODULE 1 · MICROSERVICES FOUNDATIONS

CHAPTER 01

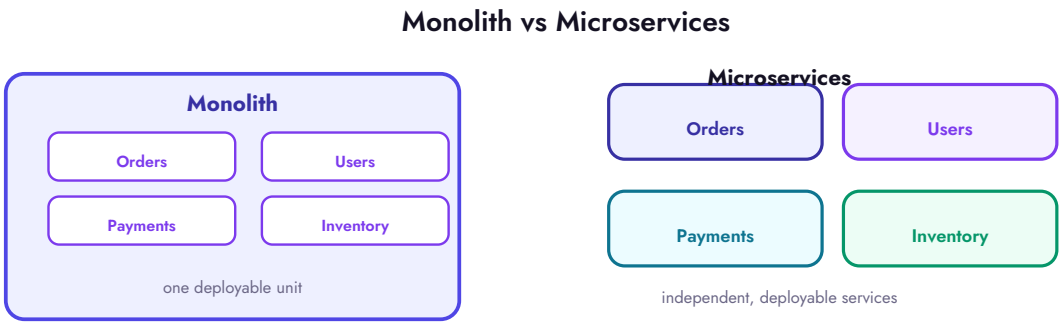
From Monolith to Microservices

What microservices are, why teams adopt them, and the trade-offs they bring.

A **microservices** architecture builds an application as a set of small, independently deployable services, each owning one business capability. It is the opposite of the **monolith** — a single deployable unit — and it trades simplicity for independence and scale.

> **Microservices**
> An architectural style structuring an application as a collection of small, loosely-coupled services, each independently deployable and owning its own data.

Monolith vs microservices



A monolith is one deployable unit; microservices split it into independent services.

Why teams adopt them

- **Independent deployment** — ship one service without redeploying everything.
- **Independent scaling** — scale only the hot services.
- **Team autonomy** — small teams own services end to end.
- **Technology freedom** — pick the right stack per service.
- **Fault isolation** — one service failing need not take down the rest.

The costs

These benefits are not free. A distributed system replaces in-process method calls with network calls that can fail, be slow, or arrive out of order — and it adds operational complexity that a monolith never had.

Gain	Cost
independent deploys	distributed complexity
independent scaling	network failure modes
team autonomy	harder end-to-end testing

© 2026 ByteMart Digital

Gain	Cost
fault isolation	data consistency challenges

WATCH OUT

Microservices solve organisational and scaling problems, not all problems. They introduce real distributed-systems complexity — adopt them because you have that scale, not because they are fashionable.

Independence at a price

Microservices trade a monolith's simplicity for independent deployment, scaling, and team ownership. Those gains are powerful at scale but come with distributed-systems complexity you must be ready to manage.

MODULE 1 · MICROSERVICES FOUNDATIONS

CHAPTER 02

Core Principles & Characteristics

The defining traits of a good microservice — loose coupling, high cohesion, and autonomy.

Not every small service is a good microservice. The style rests on a few principles: each service should be **loosely coupled**, **highly cohesive**, independently deployable, and own its own data. Get these right and the architecture works; ignore them and you build a distributed monolith.

The defining characteristics

- **Single responsibility** — one business capability per service.
- **Loose coupling** — services interact only through stable APIs.
- **High cohesion** — related behaviour lives together.
- **Independently deployable** — released on its own schedule.
- **Owns its data** — no shared database with other services.
- **Decentralised** — teams govern their own services.

Loose coupling and high cohesion

These two properties are the heart of good design. **Coupling** measures how much services depend on each other's internals; **cohesion** measures how well a service's contents belong together. Aim for low coupling and high cohesion.

Smart endpoints, dumb pipes

Microservices favour simple communication (plain HTTP or lightweight messaging) with logic in the services themselves — not in a heavy, centralised message bus that becomes a bottleneck and a coupling point.

Design for failure

In a distributed system, dependencies will fail. A well-designed service expects this — it times out, degrades gracefully, and does not cascade its failure to callers. Resilience is a first-class design concern, not an afterthought.

> **Cohesive, autonomous, resilient**
> Good microservices are loosely coupled, highly cohesive, independently deployable, and own their data. Keep pipes dumb and services smart, and design for failure from the start — these principles separate microservices from a distributed mess.

© 2026 ByteMart Digital

MODULE 1 · MICROSERVICES FOUNDATIONS

CHAPTER 03

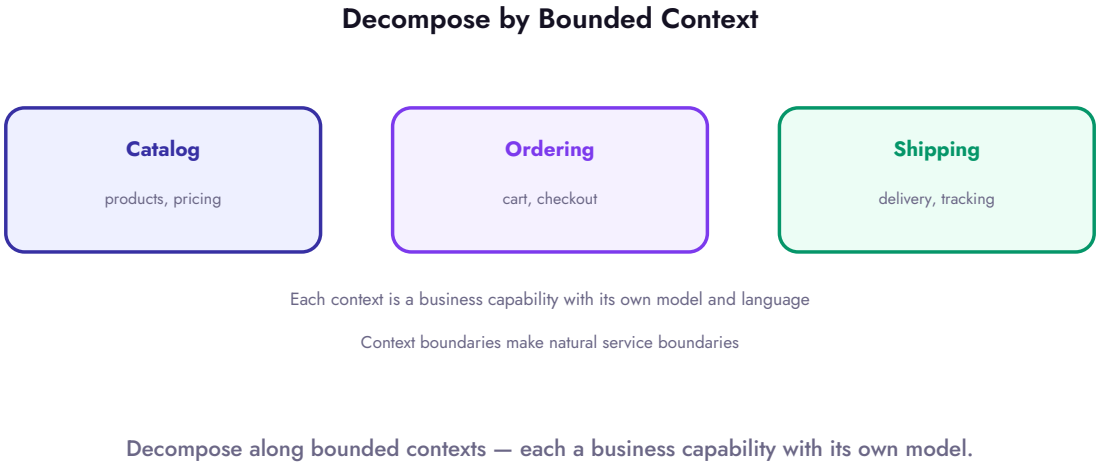
Service Decomposition

How to split a system into services using domain-driven design and bounded contexts.

The hardest question in microservices is where to draw the boundaries. Split wrong and services become chatty and tightly coupled. **Domain-Driven Design** offers the best guide: decompose along **bounded contexts** — natural divisions in the business domain.

Bounded contexts

A **bounded context** is a boundary within which a model and its language are consistent. 'Customer' means something specific in Billing and something different in Support — those are two contexts, and two candidate services.



Decompose along bounded contexts — each a business capability with its own model.

Decomposition strategies

- **By business capability** — Orders, Payments, Shipping, Catalog.
- **By bounded context** — wherever the domain language shifts.
- **By subdomain** — core, supporting, and generic subdomains.
- Avoid splitting by **technical layer** (a 'database service' is an anti-pattern).

Signs of a good boundary

- Most changes touch **one** service, not many.
- Services communicate through **coarse-grained** operations, not chatty calls.
- Each service can be understood and owned by **one team**.
- Data naturally belongs to one service, not several.

Start coarse

TIP

When unsure, start with fewer, larger services and split later. Merging is painful, but extracting a clear sub-service from a well-structured module is straightforward. Premature fine-grained splitting is a common, costly mistake.

Boundaries follow the domain

Decompose along bounded contexts and business capabilities, not technical layers. Good boundaries keep most changes local and communication coarse-grained — and when in doubt, start coarse and split as understanding grows.

© 2026 ByteMart Digital

MODULE 1 · MICROSERVICES FOUNDATIONS

CHAPTER 04

When Not to Use Microservices

Recognising when a monolith is the better choice — and the modular middle ground.

Microservices are a powerful tool for the right problem — and a costly mistake for the wrong one. Understanding when not to use them is as important as knowing how. For many applications, a well-structured monolith is the better choice.

When a monolith wins

- **Small teams** — the operational overhead outweighs the benefits.
- **Early-stage products** — boundaries are still shifting.
- **Simple domains** — not enough complexity to justify splitting.
- **Limited ops maturity** — no automation for many deployables.

The cost of premature microservices

Splitting too early locks in boundaries you do not yet understand, turning easy in-code refactors into hard cross-service migrations. You pay the full distributed-systems tax before you have the scale that repays it.

The modular monolith

A **modular monolith** is often the sweet spot: a single deployable with strong internal module boundaries. You get clean separation and simple operations now, and clear seams to extract services later if you truly need to.

- One deployable — simple to run, test, and debug.
- Enforced module boundaries — low coupling internally.
- Easy to refactor boundaries while they are still forming.
- A natural stepping stone to microservices when scale demands.

WATCH OUT

'Start with microservices' is rarely good advice for a new product. Begin with a modular monolith, learn your domain, and extract services when a real scaling or team-autonomy need appears.

The right tool, not the default

Microservices earn their complexity only at sufficient scale and team size. For small teams, young products, or simple domains, a modular monolith delivers clean structure with far less overhead — and keeps the door open to split later.

© 2026 ByteMart Digital

MODULE 2 · INTER-SERVICE COMMUNICATION

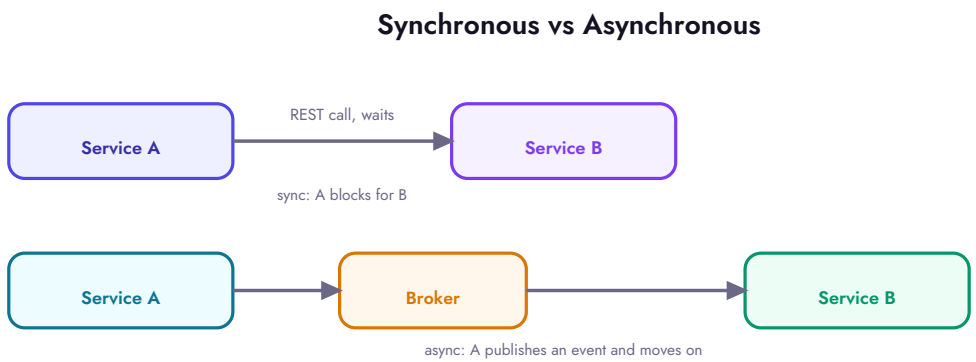
CHAPTER 05

Synchronous Communication

Request-response calls between services over REST — and their coupling and failure implications.

Services must talk to each other. The most familiar way is **synchronous** request-response over HTTP/REST: one service calls another and waits for the reply. It is simple and intuitive, but it couples the caller's availability to the callee's.

Sync vs async at a glance



Synchronous calls block for a reply; asynchronous messaging decouples services.

Calling a service

In Spring, a declarative HTTP client turns a remote call into a plain method call, keeping the calling code clean.

```java
@HttpExchange("/api/products")
public interface ProductClient {
    @GetExchange("/{id}")
    Product byId(@PathVariable Long id);
}
// injected and called like any bean:
Product p = productClient.byId(42L);
```

The coupling problem

Synchronous calls create **temporal coupling**: if the callee is down or slow, the caller is too. Chains of synchronous calls multiply this — one slow service degrades everything upstream of it.

- Simple, familiar request-response semantics.
- The caller **waits** — its latency includes the callee's.
- The caller **fails** if the callee fails, unless guarded.
- Long call chains amplify latency and failure.

© 2026 ByteMart Digital

Making sync calls safer

Never make a naked remote call. Wrap it with a timeout, a retry policy, and a circuit breaker (covered in the resilience module) so a struggling dependency cannot drag your service down.

WATCH OUT

Deep synchronous call chains are a common source of cascading failure and latency. Keep chains shallow, and consider asynchronous messaging where a response is not immediately needed.

Simple, but coupled

Synchronous REST is the easiest way for services to talk, but it couples their availability and latency. Use it for genuine request-response needs, keep chains short, and always guard remote calls against failure.

© 2026 ByteMart Digital

MODULE 2 · INTER-SERVICE COMMUNICATION

CHAPTER 06

gRPC & Efficient RPC

High-performance, contract-first communication between services with gRPC and Protocol Buffers.

When services communicate heavily, JSON over HTTP can be inefficient. **gRPC** offers a fast, strongly-typed alternative: a contract-first RPC framework using Protocol Buffers for compact binary messages and HTTP/2 for efficient transport.

Contract-first with Protobuf

You define the service and messages in a `.proto` file. Code generators produce type-safe client and server stubs in many languages from that single contract.

```text
service ProductService {
  rpc GetProduct (ProductRequest) returns (Product);
}
message ProductRequest { int64 id = 1; }
message Product { int64 id = 1; string name = 2; int32 price = 3; }
```

Why gRPC is fast

- **Binary Protobuf** — smaller and faster to parse than JSON.
- **HTTP/2** — multiplexed streams, header compression.
- **Strong contracts** — generated stubs catch mismatches at compile time.
- **Streaming** — client, server, and bidirectional streams.

REST vs gRPC

	REST/JSON	gRPC
Format	text (JSON)	binary (Protobuf)
Contract	optional (OpenAPI)	required (.proto)
Best for	public / browser APIs	internal service-to-service

When to choose which

Use REST for public and browser-facing APIs, where readability and ubiquity matter. Use gRPC for internal, high-throughput, low-latency service-to-service calls where performance and strong contracts pay off.

> **Fast, typed internal calls**
> gRPC brings compact binary messaging, HTTP/2, and generated contracts — ideal for chatty internal communication. Reach for it between services that need speed and strong typing; keep REST for public and browser-facing APIs.

© 2026 ByteMart Digital

© 2026 ByteMart Digital

MODULE 2 · INTER-SERVICE COMMUNICATION

CHAPTER 07

Asynchronous Messaging

Decoupling services with message brokers, events, and fire-and-forget communication.

Synchronous calls couple services in time. **Asynchronous messaging** breaks that coupling: a service publishes a message to a broker and moves on, and other services consume it when ready. This is the key to loosely-coupled, resilient systems.

The message broker

A **broker** — Kafka, RabbitMQ, or a cloud queue — sits between services. Producers send messages to it; consumers read from it. Neither needs to know about, or be available at the same time as, the other.

Publishing and consuming

java

```
// Producer: publish and continue
kafkaTemplate.send("orders", new OrderPlaced(orderId, total));

// Consumer: react when a message arrives
@KafkaListener(topics = "orders")
public void onOrderPlaced(OrderPlaced event) {
    inventory.reserve(event.orderId());
}
```

Messaging styles

Style	Semantics
Queue (point-to-point)	one consumer handles each message
Topic (publish-subscribe)	every subscriber gets a copy

What messaging buys you

- **Temporal decoupling** — the consumer can be down when you publish.
- **Load levelling** — the queue absorbs spikes.
- **Extensibility** — add consumers without changing producers.
- **Resilience** — messages persist until successfully processed.

WATCH OUT

Async messaging is powerful but harder to reason about: message ordering, duplicates, and failures need explicit handling. Design consumers to be idempotent so re-delivered messages do no harm.

© 2026 ByteMart Digital

Decouple in time

Asynchronous messaging lets services communicate without being available together, absorbing load and enabling new consumers freely. It is the backbone of resilient, event-driven systems — at the cost of handling ordering, duplicates, and eventual consistency.

© 2026 ByteMart Digital

MODULE 2 · INTER-SERVICE COMMUNICATION

CHAPTER 08

API Contracts & Versioning

Defining stable service contracts and evolving them without breaking consumers.

A microservice's API is a contract other teams depend on. Change it carelessly and you break them. Stable, well-versioned contracts — and backward-compatible evolution — are what let independent services change independently without constant coordination.

The contract is the interface

Each service exposes an API (REST with OpenAPI, or gRPC with Protobuf) that consumers code against. Treat it as a published, stable contract — not an implementation detail you can change at will.

Backward-compatible change

- **Add**, don't remove or rename — new optional fields are safe.
- Never change the **type** or meaning of an existing field.
- Make consumers tolerant — ignore unknown fields they receive.
- Deprecate before removing, and give consumers time to migrate.

Versioning strategies

Approach	Example
URI versioning	/api/v2/products
Header versioning	Accept: application/vnd.app.v2+json
Additive (no version)	add optional fields only

Consumer-driven contracts

Consumer-driven contract testing (e.g. Spring Cloud Contract, Pact) lets consumers specify what they expect, and verifies the provider still meets it in CI — catching breaking changes before deployment, not in production.

```java
// A contract the provider must satisfy in its build
Contract.make {
    request  { method GET(); url "/api/products/1" }
    response { status 200; body([ id: 1, name: "Pen" ]) }
}
```

Evolve without breaking

A service's API is a contract consumers rely on. Evolve it additively, version deliberately, and verify with consumer-driven contract tests — so services can change independently, which was the whole point of microservices.

© 2026 ByteMart Digital

MODULE 3 · SERVICE INFRASTRUCTURE

CHAPTER 09

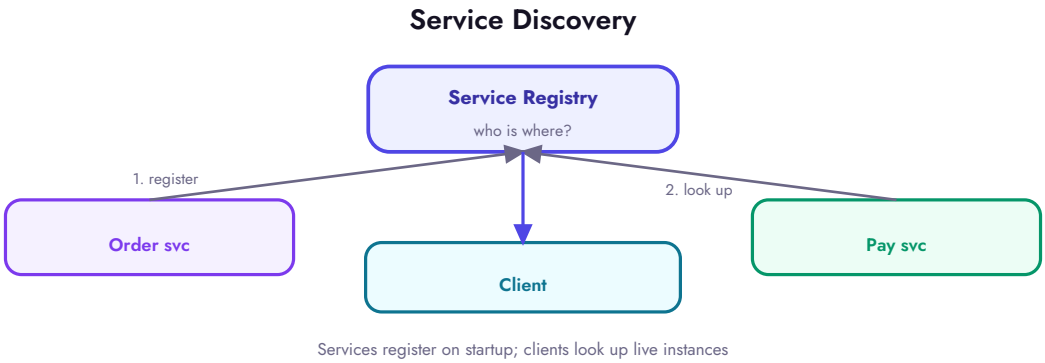
Service Discovery

Finding service instances dynamically instead of hard-coding network addresses.

In a dynamic system, service instances start, stop, and move constantly — you cannot hard-code their addresses. **Service discovery** solves this: instances register themselves, and callers look up healthy instances by name at runtime.

> **Service discovery**
> A mechanism where service instances register their location with a registry, and clients query that registry to find healthy instances by logical name.

How it works



Instances register on startup; clients look up live instances by name.

- Each instance **registers** with the registry on startup.
- It sends **heartbeats**; the registry drops instances that stop responding.
- Clients **query by name** ('order-service'), not by IP.
- The registry returns the current healthy instances.

Client-side vs server-side

Model	Who chooses the instance
Client-side	the client picks from the list
Server-side	a load balancer / gateway picks

In practice

Common registries include Netflix Eureka, Consul, and — in Kubernetes — the platform's built-in DNS-based discovery, where a service name resolves to a stable virtual address automatically.

© 2026 ByteMart Digital

TIP

On Kubernetes you often need no separate discovery tool: the platform gives every Service a stable DNS name and load-balances across its pods. Reach for Eureka or Consul mainly outside such a platform.

Find services by name

Service discovery replaces brittle hard-coded addresses with dynamic lookup: instances register and heartbeat, clients resolve by logical name. On Kubernetes this is built in; elsewhere a registry provides it.

© 2026 ByteMart Digital

MODULE 3 · SERVICE INFRASTRUCTURE

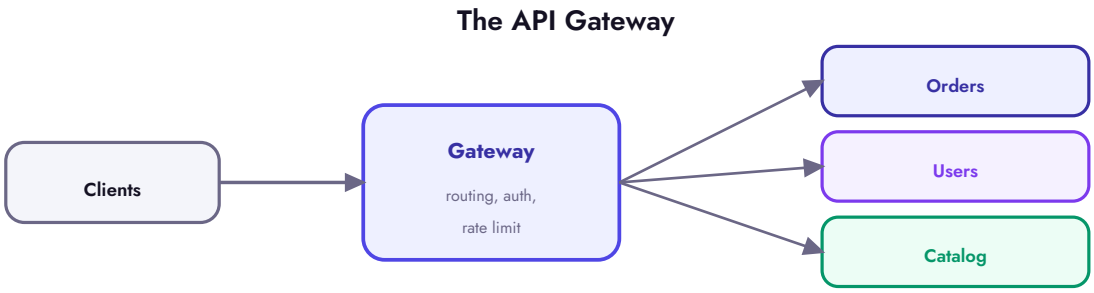
CHAPTER 10

The API Gateway

A single entry point that routes requests and handles cross-cutting concerns for clients.

Exposing dozens of services directly to clients is a mess — clients must know every address, and each service re-implements auth, rate limiting, and CORS. An **API gateway** provides one entry point that routes requests and centralises these cross-cutting concerns.

The gateway pattern



One entry point handles cross-cutting concerns and routes to services

One gateway handles cross-cutting concerns and routes requests to the right service.

What a gateway does

- **Routing** — forward each request to the right service.
- **Authentication** — verify tokens once, at the edge.
- **Rate limiting** and throttling to protect services.
- **Cross-cutting concerns** — CORS, logging, request shaping.
- **Aggregation** — optionally combine calls for a client.

Routing configuration

```text
spring:
  cloud:
    gateway:
      routes:
        - id: orders
          uri: lb://order-service      # via service discovery
          predicates:
            - Path=/api/orders/**
```

Backend for Frontend

A common variant is the **Backend for Frontend** (BFF): a dedicated gateway per client type (web, mobile), each shaping responses for that client's needs rather than forcing one API to serve all.

WATCH OUT

Do not let the gateway grow into a monolith of business logic. It should handle routing and cross-cutting concerns — keep domain logic in the services, or it becomes a bottleneck and a single point of failure.

One front door

An API gateway gives clients a single entry point and centralises routing, auth, and rate limiting so services stay focused. Keep it thin — edge concerns only — and consider a BFF per client type.

© 2026 ByteMart Digital

MODULE 3 · SERVICE INFRASTRUCTURE

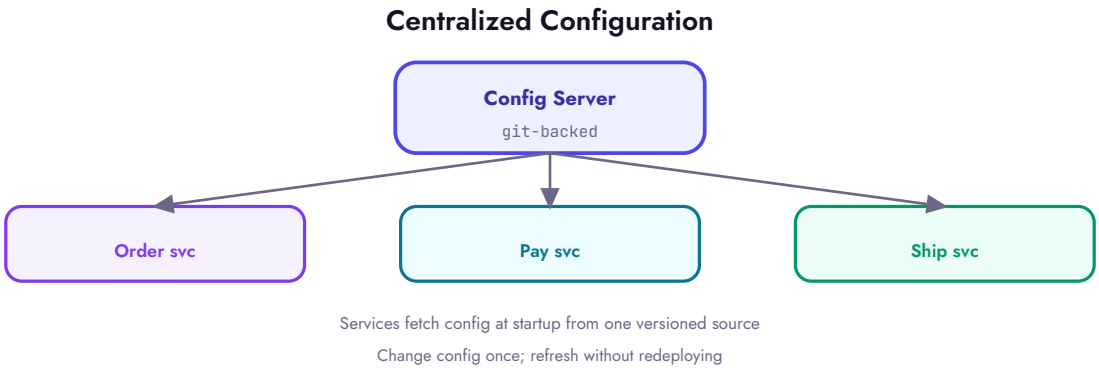
CHAPTER 11

Centralized Configuration

Managing configuration for many services from one versioned, refreshable source.

With many services across many environments, scattering configuration in each one becomes unmanageable. **Centralized configuration** keeps all settings in one versioned source that services fetch at startup — and can refresh without redeploying.

The config server



Services fetch configuration from one versioned source at startup.

A **config server** serves configuration — often backed by a Git repository — to every service. Each service asks for its configuration by name and active profile on startup.

```text
# a service points at the config server
spring:
  config:
    import: "configserver:http://config-server:8888"
  application:
    name: order-service      # fetches order-service.yml
```

Why centralise

- One versioned source of truth for all settings.
- Consistent configuration across many instances of a service.
- Change once and refresh live — no rebuild or redeploy.
- Audit trail through the backing version control.

Refreshing configuration

Services can reload changed configuration at runtime (for example via a `/actuator/refresh` endpoint or a broadcast bus), so a config change propagates without restarting anything.

Secrets

WATCH OUT

Do not store plain-text secrets in the config repository. Use a dedicated secrets manager (Vault, cloud secret stores) or encrypted values, and inject sensitive data separately from ordinary configuration.

Config in one place

Centralized configuration serves every service from one versioned, refreshable source — consistent, auditable, and changeable without redeploys. Keep secrets in a proper secrets manager, not in the config repo.

© 2026 ByteMart Digital

MODULE 3 · SERVICE INFRASTRUCTURE

CHAPTER 12

Client-Side Load Balancing

Distributing calls across service instances from the caller, using the discovery registry.

When a service has several instances, calls must be spread across them. **Client-side load balancing** lets the calling service choose an instance itself — using the discovery registry — rather than routing every call through a central load balancer.

How it works

The client fetches the list of healthy instances from the registry and picks one per call using a strategy like round-robin. There is no extra network hop through a central balancer.

```java
// 'lb://' tells the client to load-balance across instances
@Bean
@LoadBalanced
RestClient.Builder restClientBuilder() { return RestClient.builder(); }

// call by service name; an instance is chosen automatically
restClient.get().uri("lb://order-service/api/orders/1").retrieve();
```

Balancing strategies

Strategy	Picks
Round-robin	each instance in turn
Random	an instance at random
Least-connections	the least busy instance
Zone-aware	prefer the nearest zone

Client-side vs server-side

- **Client-side** — no extra hop; the caller decides.
- **Server-side** — a gateway or platform balancer decides.
- Kubernetes provides server-side balancing per Service by default.
- Client-side pairs naturally with a discovery registry like Eureka.

Spread the load at the source

Client-side load balancing uses the discovery registry to pick an instance per call, avoiding a central bottleneck. It pairs well with Eureka-style discovery; on Kubernetes, the platform's per-Service balancing often suffices.

© 2026 ByteMart Digital

MODULE 4 · RESILIENCE

CHAPTER 13

Failure in Distributed Systems

Why distributed systems fail in ways monoliths never do — and the fallacies to avoid.

In a monolith, a method call always succeeds or throws immediately. Across a network, a call can hang, time out, partially succeed, or fail in ways you never see in a single process. Building resilient microservices starts with accepting that failure is normal, not exceptional.

The fallacies of distributed computing

A classic list of false assumptions engineers make about networks. Believing any of them leads to fragile systems.

- The network is reliable — it is not; packets and connections drop.
- Latency is zero — remote calls are orders of magnitude slower.
- Bandwidth is infinite — large payloads and chatty calls cost.
- The network is secure — assume it is not.
- Topology never changes — instances come and go constantly.

New failure modes

Failure	Effect
Timeout	no response in the allotted time
Partial failure	some calls in a flow succeed, some fail
Cascading failure	one failure spreads upstream
Network partition	services cannot reach each other

Cascading failure

The most dangerous pattern: a slow dependency causes callers to pile up waiting, exhausting their threads, so they fail too — and their callers fail in turn. One struggling service can take down a whole system if nothing stops the spread.

WATCH OUT

Assume every remote call can fail or hang. Code that treats a network call like a local method — no timeout, no fallback — is the root cause of most microservices outages.

Failure is the normal case

Distributed systems fail in ways monoliths never do: timeouts, partial and cascading failures, partitions. Design every remote interaction expecting failure — the resilience patterns in this module exist precisely to contain it.

© 2026 ByteMart Digital

MODULE 4 · RESILIENCE

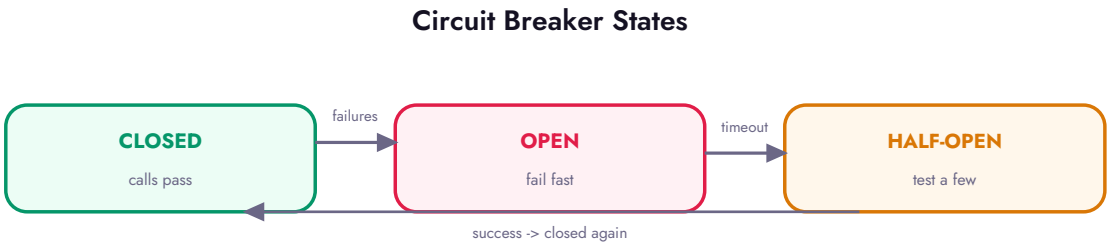
CHAPTER 14

Circuit Breakers

Stopping cascading failures by failing fast when a dependency is unhealthy.

A **circuit breaker** stops cascading failure. When a dependency starts failing, the breaker 'opens' and makes further calls fail immediately — sparing the caller from piling up on a dead service and giving the dependency room to recover.

The three states



Open the circuit on repeated failures to fail fast and let the callee recover

Closed passes calls; open fails fast; half-open tests recovery before closing.

State	Behaviour
Closed	calls pass through normally
Open	calls fail fast, no attempt made
Half-open	let a few through to test recovery

With Resilience4j

In Spring, a circuit breaker wraps a call and provides a **fallback** to run when the circuit is open — returning cached data, a default, or a graceful error.

```java
@CircuitBreaker(name = "catalog", fallbackMethod = "cachedProduct")
public Product getProduct(Long id) {
    return catalogClient.byId(id);       // real call
}

public Product cachedProduct(Long id, Throwable t) {
    return cache.getOrDefault(id, Product.unavailable());   // fallback
}
```

Tuning the breaker

- **Failure threshold** — the error rate that trips it open.

- **Wait duration** — how long to stay open before testing.
- **Sliding window** — how many calls are measured.
- A good **fallback** is what turns a failure into graceful degradation.

Fail fast, recover gracefully

A circuit breaker detects a failing dependency and short-circuits calls to it, preventing cascading failure and giving it time to heal. Paired with a sensible fallback, it turns an outage into graceful degradation.

© 2026 ByteMart Digital

MODULE 4 · RESILIENCE

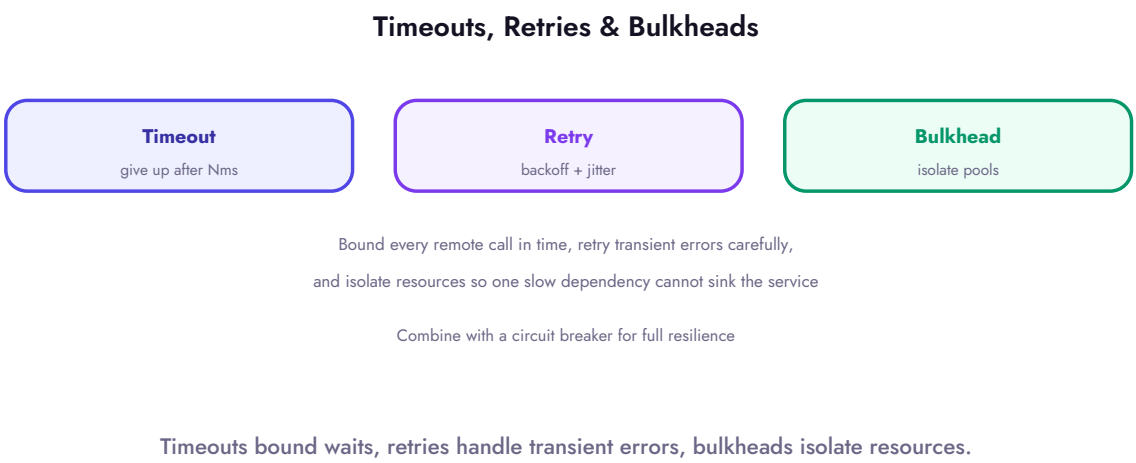
CHAPTER 15

Retries, Timeouts & Bulkheads

The core resilience toolkit for bounding, retrying, and isolating remote calls.

The circuit breaker works alongside three more patterns. **Timeouts** bound how long you wait, **retries** recover from transient blips, and **bulkheads** isolate resources so one failing dependency cannot sink the whole service.

The toolkit



Timeouts bound waits, retries handle transient errors, bulkheads isolate resources.

Timeouts

Every remote call must have a timeout. A call with no timeout can hang forever, tying up a thread and eventually the whole service. Set timeouts based on the dependency's expected latency, not on hope.

Retries — carefully

Retrying can recover from a transient glitch, but naive retries amplify load and can worsen an outage. Use **exponential backoff** with **jitter**, cap the attempts, and only retry **idempotent** operations.

```java
@Retry(name = "catalog")          // backoff configured externally
@TimeLimiter(name = "catalog")    // enforce a timeout
public CompletableFuture<Product> getProduct(Long id) {
    return catalogClient.byIdAsync(id);
}
```

Bulkheads

Named after a ship's watertight compartments, a **bulkhead** limits how many concurrent calls a dependency may consume. If that dependency stalls, only its bulkhead fills — the rest of the service keeps working.

© 2026 ByteMart Digital

WATCH OUT

Retrying a non-idempotent operation (like 'charge card') can double-execute it. Retry only idempotent calls, or make operations idempotent with a request/idempotency key so duplicates are harmless.

Bound, retry, isolate

Timeouts stop calls hanging, careful retries with backoff recover from blips, and bulkheads contain a failing dependency. Combined with a circuit breaker, they form the standard toolkit for surviving in a world where remote calls fail.

© 2026 ByteMart Digital

MODULE 4 · RESILIENCE

CHAPTER 16

Rate Limiting & Backpressure

Protecting services from overload by limiting request rates and signalling capacity.

A service that accepts unlimited traffic will eventually be overwhelmed. **Rate limiting** caps how many requests a caller may make, and **backpressure** signals when a system is at capacity — together protecting services from overload and abuse.

Rate limiting

A rate limiter rejects or delays requests beyond an allowed rate, protecting downstream capacity and ensuring fair use. It is commonly enforced at the API gateway, per client or API key.

Algorithm	Idea
Token bucket	spend tokens; refill at a fixed rate
Leaky bucket	process at a steady drain rate
Fixed window	N requests per time window
Sliding window	smoother rolling-window count

Responding to limits

When a limit is exceeded, return `429 Too Many Requests` with a `Retry-After` header so well-behaved clients know to back off and when to try again.

```text
HTTP/1.1 429 Too Many Requests
Retry-After: 30
{ "error": "rate_limit_exceeded" }
```

Backpressure

Backpressure is a system telling upstream producers to slow down when it cannot keep up — through bounded queues, blocking, or reactive streams — rather than silently buffering until it runs out of memory.

- Bounded queues reject or block when full, instead of growing unboundedly.
- Reactive streams propagate demand so producers respect consumer capacity.
- Load shedding drops low-priority work under extreme load.
- Together they keep a system stable instead of collapsing under a spike.

> **Protect capacity deliberately**
> Rate limiting caps incoming load per client and backpressure signals when the system is full. Enforce limits at the gateway, respond with 429 and Retry-After, and bound queues — so overload degrades gracefully instead of crashing.

© 2026 ByteMart Digital

MODULE 5 · DISTRIBUTED DATA

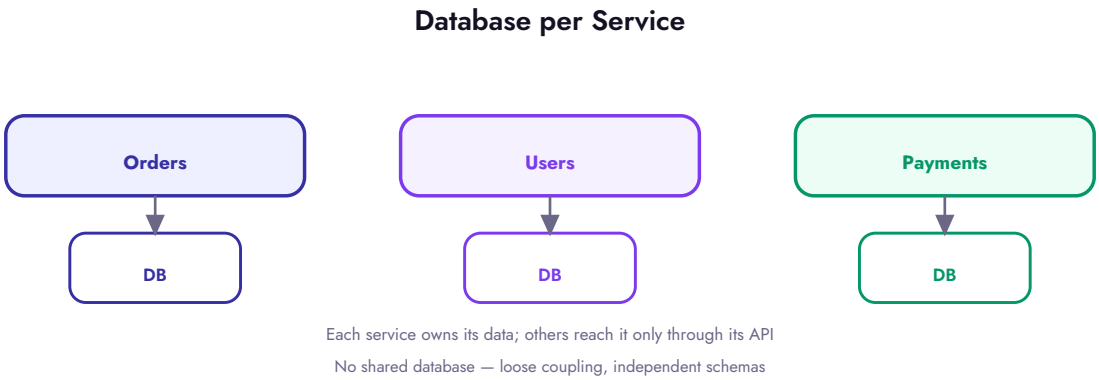
CHAPTER 17

Database per Service

Why each service owns its data privately — and the consequences for queries and consistency.

The single most important data rule in microservices: each service owns its data, and no other service touches its database directly. This **database-per-service** pattern preserves loose coupling — but it means data is now scattered, which reshapes how you query and stay consistent.

The pattern



Each service owns its database; others reach its data only through its API.

Each service has a private data store that only it accesses. Other services get that data by calling the owning service's API or by subscribing to its events — never by querying its tables.

Why not a shared database

- A shared schema **couples** services — one team's change breaks others.
- It prevents independent deployment and independent scaling.
- It lets services depend on each other's internal data shapes.
- Private data is the foundation of true service autonomy.

The consequences

Splitting data creates two hard problems that the rest of this module addresses: **queries that span services** (you can no longer JOIN across databases) and **transactions that span services** (no single database transaction covers them all).

Lost	Replaced by
cross-service JOIN	API composition or CQRS views
single ACID transaction	sagas + eventual consistency

WATCH OUT

The moment two services share a database, you no longer have microservices — you have a distributed monolith with all the costs and none of the independence. Keep every data store private.

Own your data, privately

Database-per-service is what makes services independent: private data, accessed only through APIs and events. It costs you cross-service JOINs and single transactions — which sagas, events, and CQRS then help you replace.

© 2026 ByteMart Digital

MODULE 5 · DISTRIBUTED DATA

CHAPTER 18

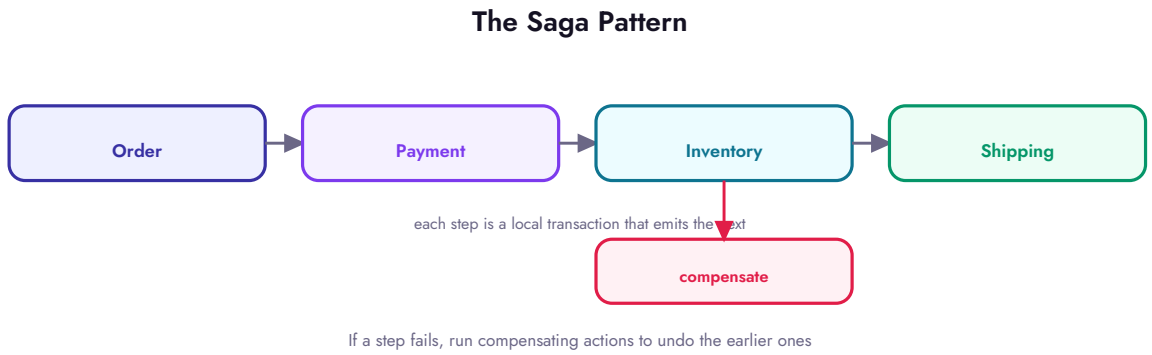
The Saga Pattern

Coordinating a transaction across services with local steps and compensating actions.

With data split across services, a single ACID transaction can no longer span a business process like 'place order'. The **saga pattern** solves this: model the process as a sequence of local transactions, each triggering the next, with **compensating actions** to undo on failure.

> **Saga**
> A sequence of local transactions across services where each step publishes an event triggering the next, and failures are handled by running compensating transactions that undo prior steps.

How a saga runs



Each step is a local transaction; a failure triggers compensating actions to undo earlier ones.

Compensation, not rollback

There is no global rollback across services. Instead, each step has a **compensating action** that semantically undoes it — refund the payment, release the reserved stock — run in reverse order when a later step fails.

Two coordination styles

Style	Coordination
Choreography	each service reacts to events (no central brain)
Orchestration	a coordinator directs each step

- **Choreography** — decentralised, simple for short sagas, harder to trace.
- **Orchestration** — a central saga manager; clearer for complex flows.
- Steps and compensations must be **idempotent** (they may retry).
- The system is **eventually consistent** during the saga.

© 2026 ByteMart Digital

WATCH OUT

Compensation is not a true rollback — other services may have already seen the intermediate state. Design compensations to be semantically correct, and accept that sagas expose brief, eventually-consistent windows.

Distributed transactions, undone by hand

A saga replaces the impossible cross-service ACID transaction with local steps plus compensations. Choose choreography or orchestration, make every step idempotent, and accept eventual consistency — this is how business processes span services safely.

© 2026 ByteMart Digital

MODULE 5 · DISTRIBUTED DATA

CHAPTER 19

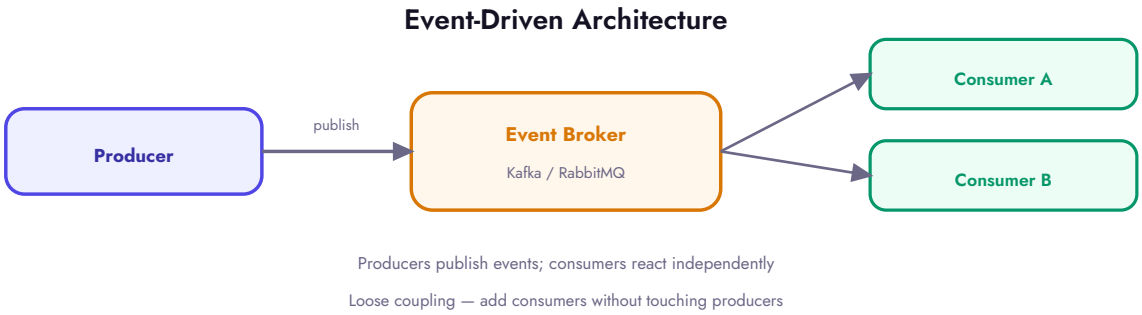
Event-Driven Architecture & CQRS

Communicating state changes through events, and separating read and write models.

Events are how services stay in sync without tight coupling. In an **event-driven architecture**, services publish facts about what happened, and others react. **CQRS** builds on this by separating the write model from tailored read models.

Events as facts

An event records something that happened — `OrderPlaced`, `PaymentReceived`. Services publish events; interested services subscribe and update their own state. Producers do not know or care who consumes.



Producers publish events; consumers react independently, staying loosely coupled.

CQRS

Command Query Responsibility Segregation separates the model that **changes** data (commands) from the model that **reads** it (queries). Read-optimised views are built from events, letting each side scale and evolve independently.

- **Command side** — validates and applies changes, emits events.
- **Query side** — maintains denormalised views for fast reads.
- Views are updated by consuming the command side's events.
- Solves cross-service queries: build a local read model from events.

Event sourcing

A related pattern, **event sourcing**, stores state as the full sequence of events rather than just the current value. You can rebuild state by replaying events and audit exactly how it changed — powerful, but more complex.

WATCH OUT

Event-driven systems trade immediate consistency for decoupling. Consumers see changes slightly later, and you must handle out-of-order and duplicate events. Adopt CQRS and event sourcing where the benefits are real — not everywhere.

© 2026 ByteMart Digital

React to what happened

Events let services share state changes without coupling; CQRS uses them to build fast, purpose-built read models that also answer cross-service queries. Powerful patterns — apply them deliberately, accepting eventual consistency.

© 2026 ByteMart Digital

MODULE 5 · DISTRIBUTED DATA

CHAPTER 20

Eventual Consistency

Trading immediate consistency for availability, and designing systems that converge.

Distributed data forces a fundamental trade-off. You usually cannot have immediate, global consistency and high availability across a network. Microservices lean toward **eventual consistency**: data becomes consistent over time rather than instantly — a shift that changes how you design.

The CAP trade-off

The **CAP theorem** says that when a network partition happens, a system must choose between consistency and availability. Most microservices choose availability, accepting that different services may briefly disagree.

Model	Guarantee
Strong consistency	everyone sees the latest value now
Eventual consistency	all converge, given time, no new writes

Designing for it

- Make operations **idempotent** so retries and duplicates are safe.
- Use **unique keys** to detect and ignore duplicate events.
- Tolerate **out-of-order** events; use versions or timestamps.
- Communicate temporary inconsistency in the UI where it matters.

When you truly need strong consistency

Some operations — a financial ledger, stock that must never oversell — need stronger guarantees. Keep those within a single service and one database transaction, so their invariant is enforced locally rather than across services.

TIP

Do not fight eventual consistency everywhere. Embrace it for cross-service data, and keep the few genuinely strong-consistency invariants inside a single service's boundary where a real transaction can enforce them.

Converge, don't insist on now

Across services, immediate global consistency is usually impossible — design for eventual consistency with idempotency, deduplication, and out-of-order tolerance. Reserve strong consistency for invariants you can keep inside one service.

© 2026 ByteMart Digital

MODULE 6 · OBSERVABILITY

CHAPTER 21

Centralized Logging

Aggregating logs from many services into one searchable place, correlated by request.

In a monolith, logs live in one file. Across dozens of services and hundreds of instances, logs are scattered everywhere — useless unless aggregated. **Centralized logging** ships every service's logs to one searchable system, correlated so you can follow a single request.

The aggregation pipeline

Each service writes structured logs; an agent ships them to a central store where they are indexed and searchable. Common stacks are the ELK/Elastic stack, Loki with Grafana, or a cloud logging service.

- Services log to stdout in **structured (JSON)** form.
- A collector ships logs to a central, indexed store.
- Engineers search across **all** services in one place.
- Dashboards and alerts are built on the aggregated logs.

Correlation IDs

The key to tracing a request across services is a **correlation id**: generate it at the gateway, propagate it through every call, and include it in every log line. Then one search reconstructs the request's whole journey.

```text
# every log line carries the same correlation id
2026-01-10 INFO  [traceId=7af3c1] order-service   received order 88
2026-01-10 INFO  [traceId=7af3c1] payment-service charged 499
2026-01-10 WARN  [traceId=7af3c1] shipping-service address incomplete
```

Logging discipline

WATCH OUT

Never log secrets, tokens, or full personal data — aggregated logs are widely accessible and long-lived. Log meaningful business events with context, and keep noisy DEBUG output out of production.

One place, one request's story

Centralized logging turns scattered per-instance logs into a single searchable system, and a propagated correlation id lets you follow one request across every service. Log structured, correlated, secret-free events — it is your first debugging tool.

© 2026 ByteMart Digital

MODULE 6 · OBSERVABILITY

CHAPTER 22

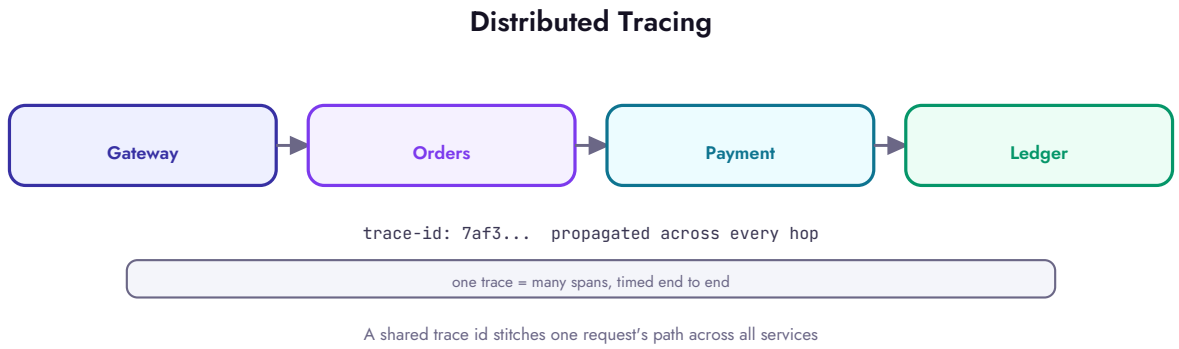
Distributed Tracing

Following a single request across services to find where time and errors occur.

When a request touches five services, and it is slow, which service is to blame? Logs alone cannot show the flow. **Distributed tracing** follows a single request across every service, timing each hop — turning a mystery into a clear picture of where time and errors go.

Traces and spans

A **trace** represents one request's full journey. It is made of **spans** — one per operation or service hop — each timed and linked to its parent, all sharing a single trace id.



A shared trace id links spans across services, timing one request end to end.

How it works

- The trace id is created at the entry point and **propagated** in headers.
- Each service records a **span** with start, end, and metadata.
- Spans are exported to a tracing backend (Zipkin, Jaeger, Tempo).
- The backend reconstructs the full timed tree of the request.

In Spring

Micrometer Tracing (with an exporter) instruments Spring applications automatically — propagating context across REST clients, messaging, and async boundaries with little to no code.

What tracing reveals

- The **slow hop** in a chain — exactly which service or call.
- Unexpected **fan-out** or N+1 remote calls.
- Where an **error** originated in a multi-service flow.
- The real end-to-end **latency** a user experiences.

© 2026 ByteMart Digital

See the whole journey

Distributed tracing stitches one request's path across services into a timed tree of spans, pinpointing slow hops and error origins that logs alone cannot. With Micrometer Tracing it is largely automatic — and indispensable for debugging distributed flows.

© 2026 ByteMart Digital

MODULE 6 · OBSERVABILITY

CHAPTER 23

Metrics & Health Monitoring

Measuring service health with metrics, the golden signals, and actionable alerts.

Logs tell you what happened; **metrics** tell you how the system is behaving right now and over time. Aggregated metrics, sensible dashboards, and alerts on the signals users actually feel are what let you run many services without watching them constantly.

Metrics across services

Each service exposes metrics (via Micrometer and Actuator) that a system like Prometheus scrapes and stores. Grafana dashboards then visualise the fleet's health at a glance.

The golden signals

Signal	Question
Latency	how long do requests take?
Traffic	how much demand is there?
Errors	how many requests fail?
Saturation	how full is the system?

Health checks

Each service exposes **liveness** and **readiness** endpoints. The orchestrator restarts instances that fail liveness and withholds traffic from those not ready — automated, per-instance health management.

```text
GET /actuator/health/liveness   -> UP (restart if DOWN)
GET /actuator/health/readiness  -> UP (route traffic if UP)
```

Alerting well

- Alert on **symptoms users feel** (errors, latency), not raw internals.
- Make every alert **actionable** — no alert without a response.
- Avoid alert fatigue; tune thresholds to real problems.
- Track service-level objectives (SLOs) and alert on burning through them.

Know the system's health

Metrics reveal how services behave in aggregate; the golden signals — latency, traffic, errors, saturation — are what to watch. Combine them with automated health checks and actionable, symptom-based alerts to run a fleet with confidence.

© 2026 ByteMart Digital

MODULE 7 · SECURITY & DEPLOYMENT

CHAPTER 24

Securing Microservices

Authenticating at the edge and propagating identity with OAuth2 and JWT across services.

Security in microservices is harder than in a monolith: many services, many network hops, and no single session. The standard approach authenticates once at the edge, issues a signed token, and propagates that verified identity to every downstream service.

Token-based identity

A user authenticates against an **identity provider** (OAuth2 / OpenID Connect) and receives a signed **JWT**. The gateway validates it, and the token — carrying identity and roles — flows with each internal call.

- Authenticate once at the **edge** (gateway or identity provider).
- Issue a signed, short-lived **JWT** with identity and scopes.
- Propagate the token on internal calls; each service verifies it.
- Services authorise locally based on the token's claims.

Verifying tokens downstream

java

```
// each service validates the JWT signature and reads claims
http.oauth2ResourceServer(oauth -> oauth.jwt(Customizer.withDefaults()));

@GetMapping("/orders")
@PreAuthorize("hasAuthority('SCOPE_orders.read')")
public List<Order> myOrders(@AuthenticationPrincipal Jwt jwt) {
    return service.forUser(jwt.getSubject());
}
```

Service-to-service security

Beyond user identity, services should trust each other's calls. **mTLS** (mutual TLS) authenticates both ends of a connection; a service mesh can enforce it transparently for all internal traffic.

Defence in depth

- Do not rely only on the gateway — each service verifies tokens too.
- Keep tokens **short-lived**; use refresh tokens for longer sessions.
- Apply least privilege with fine-grained **scopes**.
- Never trust the network; secure internal traffic with mTLS.

Verify once, propagate, re-check
Authenticate at the edge, carry a signed JWT through the system, and have each service verify it and authorise locally. Add mTLS for service-to-service trust — security in a distributed system is layered, not gate-only.

© 2026 ByteMart Digital

MODULE 7 · SECURITY & DEPLOYMENT

CHAPTER 25

Containerization

Packaging each service as a portable container image that runs identically everywhere.

Microservices and containers go together. A **container** packages a service with its runtime and dependencies into a portable image that runs identically on any machine — solving the 'works on my machine' problem and giving every service a uniform deployment unit.

Why containers fit microservices

- **Isolation** — each service runs in its own container.
- **Portability** — the same image runs on laptop, CI, and prod.
- **Consistency** — identical runtime everywhere.
- **Uniformity** — every service deploys the same way, whatever its stack.

A service image

text

```
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/order-service.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Image best practices

- Use small, current base images; run as a **non-root** user.
- Layer dependencies separately from code for cache-friendly builds.
- Tag images by **version**; deploy immutable artifacts.
- Configure via environment variables, not baked-in files.

TIP

You do not always need a hand-written Dockerfile: Spring Boot's build plugin and tools like Jib or Buildpacks produce optimised, layered images directly from your build, applying good defaults automatically.

One uniform deployment unit

Containers package each service with its runtime into a portable, immutable image that runs the same everywhere. They give a polyglot microservices fleet one consistent deployment unit — the foundation for orchestration.

© 2026 ByteMart Digital

MODULE 7 · SECURITY & DEPLOYMENT

CHAPTER 26

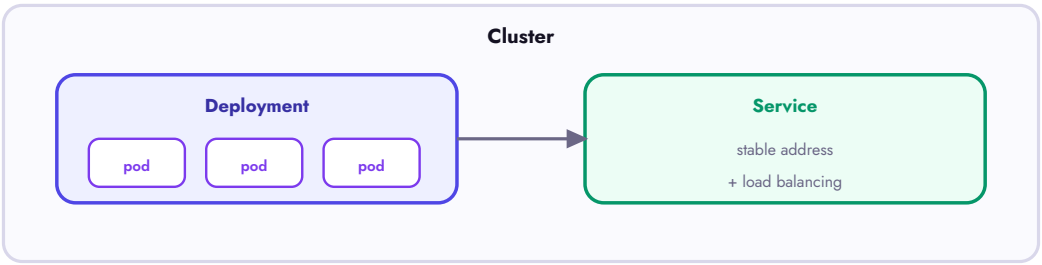
Orchestration with Kubernetes

Running, scaling, and healing many containerized services with Kubernetes.

Running one container is easy; running hundreds across many machines — scaling, restarting, networking, and updating them — needs an orchestrator. **Kubernetes** is the standard: you declare the desired state, and it continuously makes reality match.

The core objects

Kubernetes at a Glance



Deployments keep pods running; Services give them a stable endpoint

Deployments keep pods running; Services give them a stable, load-balanced endpoint.

Object	Role
Pod	one or more containers, the unit of running
Deployment	keeps N replicas of a pod running
Service	stable address + load balancing for pods
ConfigMap / Secret	configuration and secrets

Declarative desired state

You describe what you want — 'run three replicas of order-service' — and Kubernetes reconciles continuously. If a pod dies, it starts a new one; if a node fails, it reschedules the pods elsewhere.

```text
kind: Deployment
spec:
  replicas: 3
  template:
    spec:
      containers:
        - name: order-service
          image: bytemart/order-service:1.4
          readinessProbe:
            httpGet: { path: /actuator/health/readiness, port: 8080 }
```

What Kubernetes provides

- **Self-healing** — restarts failed containers, reschedules pods.
- **Scaling** — manual or automatic based on load.
- **Service discovery** and load balancing built in.
- **Rolling updates** and rollbacks with health checks.

Declare desired state, let it reconcile

Kubernetes runs and heals a fleet of containers from a declarative spec: Deployments keep replicas alive, Services expose them, and the platform handles discovery, scaling, and updates. It is the operating system for microservices at scale.

MODULE 7 · SECURITY & DEPLOYMENT

CHAPTER 27

CI/CD & Deployment Strategies

Automating build and release, and deploying safely with rolling, blue-green, and canary strategies.

Independent deployment is a core promise of microservices — but only if releasing is safe and automated. **CI/CD** pipelines build and ship each service automatically, and **deployment strategies** roll out changes without downtime or big-bang risk.

The pipeline

Continuous integration builds and tests every change; continuous delivery packages and deploys it. Each service has its own pipeline, so teams release independently.

- **Build** — compile and run unit tests on every commit.
- **Test** — integration and contract tests catch breakage.
- **Package** — produce a versioned container image.
- **Deploy** — promote through environments automatically.

Deployment strategies

Strategy	How it releases
Rolling	replace instances gradually
Blue-green	switch traffic to a new full stack
Canary	send a small % to the new version first

Canary and rollback

A **canary** release exposes the new version to a small slice of traffic while you watch its metrics. If errors rise, roll back before most users are affected; if healthy, ramp up gradually to 100%.

Safe release habits

- Deploy **immutable**, versioned artifacts — never mutate in place.
- Keep changes small and frequent to shrink blast radius.
- Use **feature flags** to decouple deploy from release.
- Automate rollback and watch metrics during every rollout.

Ship often, ship safely

CI/CD automates build-to-deploy per service so teams release independently, and rolling, blue-green, and canary strategies make each release low-risk. Small, frequent, automated deploys with fast rollback are what make microservice independence real.

© 2026 ByteMart Digital

MODULE 8 · PATTERNS & PRACTICE

CHAPTER 28

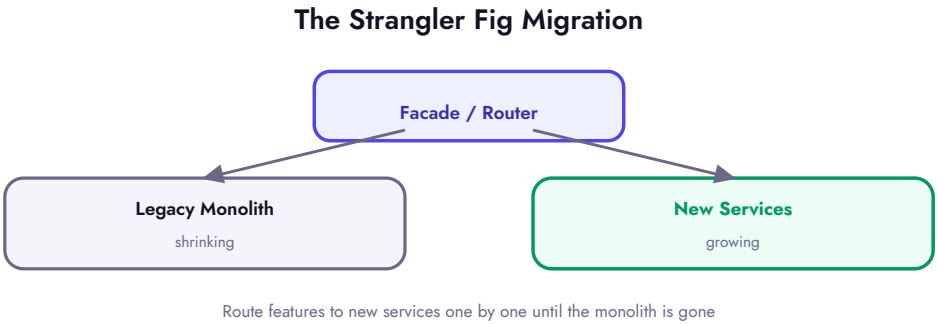
Key Microservices Patterns

The recurring design patterns — strangler fig, sidecar, BFF, and more — that solve common problems.

Certain problems recur across microservice systems, and the community has named proven solutions. Knowing these patterns gives you a shared vocabulary and a head start — you recognise the shape of a problem and reach for the established answer.

Strangler fig: migrating safely

The **strangler fig** pattern migrates a monolith to services incrementally. A facade routes requests, sending new or rewritten features to services while the monolith shrinks — no risky big-bang rewrite.



Route features to new services one by one until the monolith is fully replaced.

Structural patterns

Pattern	Solves
API Gateway	single entry point, edge concerns
Backend for Frontend	client-specific APIs
Sidecar	attach shared capabilities to a service
Service Mesh	networking, security, observability

Sidecar and service mesh

A **sidecar** runs alongside a service in the same pod, adding capabilities — proxying, logging, security — without changing the service. A **service mesh** is a fleet of sidecars managing all service-to-service traffic, handling retries, mTLS, and tracing transparently.

Data patterns recap

- **Database per service** — private data for autonomy.
- **Saga** — transactions across services via compensation.
- **CQRS** — separate read and write models.

© 2026 ByteMart Digital

- **API composition** — answer cross-service queries by combining calls.

Named solutions to common shapes

Patterns like strangler fig, sidecar, BFF, and service mesh package proven answers to recurring microservice problems. Learn them as vocabulary and toolkit — they turn hard architectural decisions into recognised, well-understood choices.

© 2026 ByteMart Digital

MODULE 8 · PATTERNS & PRACTICE

CHAPTER 29

Testing Microservices

Testing across service boundaries with the test pyramid, contract tests, and end-to-end checks.

Testing a distributed system is harder than testing a monolith — you cannot just run everything together cheaply. A layered strategy, anchored by fast unit tests and **contract tests** between services, keeps confidence high without slow, brittle end-to-end suites.

The test pyramid, distributed

Level	Scope
Unit	one class, mocked collaborators
Integration	a service with its database
Contract	the agreement between two services
End-to-end	a flow across several services

Contract testing

Consumer-driven contract testing is the key technique for microservices. The consumer defines what it expects from a provider; the provider verifies it still meets that contract in its own build — catching breaking changes without running both services together.

```java
// Provider's build verifies it satisfies the consumer's contract
@Test
void honorsProductContract() {
    // generated from the contract: GET /products/1 -> 200 {id,name}
    // fails the build if the provider's response no longer matches
}
```

Testing boundaries

- Mock external services in unit and integration tests.
- Use **Testcontainers** to test against a real database or broker.
- Rely on **contracts** instead of spinning up every dependency.
- Keep **end-to-end** tests few — they are slow and flaky.

WATCH OUT

Do not try to test everything end-to-end. Large end-to-end suites across many services are slow, brittle, and hard to debug. Push confidence down to fast unit and contract tests, and keep end-to-end coverage thin and focused.

© 2026 ByteMart Digital

Confidence without full assembly

A distributed test strategy leans on many fast unit tests and consumer-driven contract tests to verify service agreements cheaply, reserving slow end-to-end tests for a few critical flows. Contracts are what let services be tested — and released — independently.

© 2026 ByteMart Digital

MODULE 8 · PATTERNS & PRACTICE

CHAPTER 30

Anti-Patterns & Best Practices

The traps that sink microservice projects, and the habits that keep them healthy.

Most microservice failures are not technical accidents — they are known anti-patterns repeated. This closing chapter gathers the traps to avoid and the practices, drawn from the whole guide, that keep a microservices architecture healthy as it grows.

Anti-patterns to avoid

- **Distributed monolith** — services so coupled they must deploy together.
- **Shared database** — the fastest way to lose all independence.
- **Chatty services** — too many fine-grained synchronous calls.
- **Nano-services** — splitting so finely that overhead dwarfs the work.
- **No resilience** — naked remote calls that cascade on failure.
- **Premature adoption** — microservices before you have the scale.

Best practices

- Split along **bounded contexts**; keep services cohesive and coarse.
- Give every service its **own data**; integrate via APIs and events.
- Guard every remote call with timeouts, retries, and circuit breakers.
- Automate everything: build, test, deploy, and rollback.
- Invest early in **observability** — logs, traces, and metrics.

Organisational alignment

By **Conway's Law**, your architecture will mirror your team structure. Microservices work when small, autonomous teams own services end to end. Without that alignment, service boundaries fight the org chart and coordination overhead returns.

The pragmatic path

TIP

Start with a modular monolith, invest in automation and observability before you split, and extract services only where a real scaling or team-autonomy need exists. Let the architecture grow with the problem, not ahead of it.

Avoid the known traps

Microservices succeed through discipline: cohesive boundaries, private data, resilience, automation, observability, and team alignment. Avoid the distributed monolith and premature splitting — grow the architecture to fit the problem, and it will serve you well.

© 2026 ByteMart Digital